



Pi Panel Documentation Guide

Raspberry Pi 5, Pimoroni Explorer HAT Pro,
GPIO LED control, and thermistor
temperature measurement

Project: pi-panel

Platform: Raspberry Pi 5 / Qt 6 / C++20

Updated: September 2026

Table of Contents

01	System overview	3
02	Build, test, and launch	4
03	LED control	5
04	Explorer HAT button input	6
05	Thermistor measurement	7
06	Temperature calibration	8
07	Error handling and troubleshooting	9
08	Configuration reference	10

This guide describes the checked-in implementation. The Markdown files in the `documentation` folder remain the editable feature references.

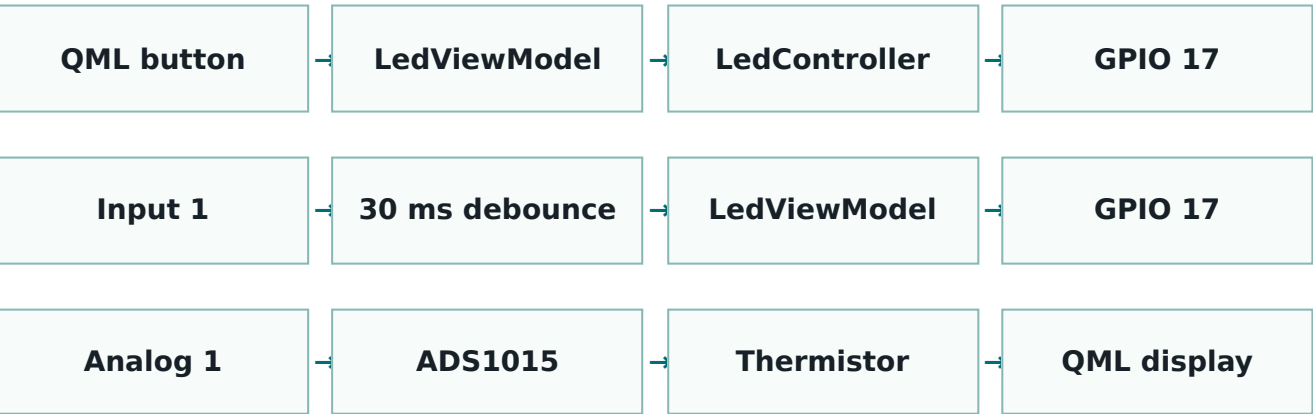
1. System Overview

Pi Panel is a Qt Quick application that controls Raspberry Pi GPIO and reads an analog thermistor through a Pimoroni Explorer HAT Pro. C++ owns hardware access and application state; QML renders the window.

Primary components

Component	Responsibility
<code>main.cpp</code>	Creates services, the view model, signal handling, and polling timers.
<code>GpiodService</code>	Requests and accesses GPIO lines through <code>libgpiod</code> .
<code>LedController</code>	Tracks LED state and interprets the button input.
<code>Ads1015</code>	Reads Explorer HAT analog channels over Linux I2C.
<code>Thermistor</code>	Converts divider voltage into degrees Fahrenheit.
<code>LedViewModel</code>	Exposes LED, voltage, temperature, and error properties to QML.
<code>Main.qml</code>	Displays controls, measurements, and errors.

Runtime paths



The GPIO timer runs every 5 ms. It checks shutdown requests, samples Explorer HAT Input 1, and sends stable presses through the view model. A separate 200 ms timer samples voltage and temperature.

2. Build, Test, and Launch

Build and test

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --config Debug
ctest --test-dir build --output-on-failure
```

VS Code debug profiles

Profile	Destination	Requirements
Debug pi_panel on Pi desktop	Pi monitor at <code>DISPLAY=:0</code>	Graphical desktop logged in as <code>bill</code> .
Debug pi_panel over forwarded X11	Windows through XLaunch	XLaunch and a connected PuTTY session with X11 forwarding.

Pi desktop

1. Log in to the Pi graphical desktop as `bill`.
2. Connect VS Code to the Pi and open the project.
3. Select **Debug pi_panel on Pi desktop**.
4. Press F5. The build task runs and the window opens on the Pi.

Forwarded X11

1. Start XLaunch on Windows.
2. Connect through PuTTY with X11 forwarding enabled.
3. Run `echo $DISPLAY` in PuTTY.
4. Make the forwarded profile's `DISPLAY` value match, select that profile, and press F5 twice if it pauses at entry.

Resource ownership: Stop any existing Pi Panel process before starting another instance. GPIO lines can only be requested by one process at a time.

3. LED Control

Hardware mapping

The default LED is BCM GPIO 17, which is LED 2 on the Explorer HAT Pro. `GpiodService` requests it as an output on `/dev/gpiochip0` and initializes it inactive.

Software flow

1. The user presses **Toggle LED** or the debounced physical button fires.
2. `LedViewModel::toggle()` calls `LedController::toggle()`.
3. The controller writes `ACTIVE` or `INACTIVE` through `IGpioService`.
4. The controller updates its stored state.
5. The view model emits `ledOnChanged`.
6. QML updates the indicator and label.

Shutdown behavior

Normal application exit calls `turnOff()`. The controller destructor also attempts to turn the LED off while suppressing hardware exceptions. A forced process termination can bypass normal cleanup, so shutdown behavior must not be treated as a hardware safety mechanism.

4. Explorer HAT Button Input

Explorer HAT Input 1 maps to BCM GPIO 23. The HAT input passes through an onboard buffer, so Raspberry Pi internal pull resistors cannot bias the external terminal. The application requests the line as an input with no internal bias and interprets high as pressed.

Recommended wiring

```
3.3 V or 5 V ---- push button ---- Input 1
                                   |
                                   10 kΩ
                                   |
                                   GND
```

Do not leave Input 1 floating. The external 10 kΩ pull-down holds the input low while released. Pressing the button drives it high. Explorer HAT digital inputs are 5 V tolerant.

Debouncing

The application samples the button every 5 ms. `ButtonDebouncer` requires a new reading to remain stable for 30 ms. A stable press toggles once; release updates the accepted state but does not toggle.

5. Thermistor Measurement

Voltage divider

```
3.3 V
|
+-- 10 kΩ fixed resistor
|
+-- Explorer HAT Pro Analog 1
|
+-- 10 kΩ NTC thermistor
|
GND
```

The Explorer HAT Pro ADS1015 uses I2C address `0x48`. Pimoroni maps Analog 1 to ADC channel 3. At the thermistor's nominal 25°C point, both resistances are about 10 kΩ and the voltage is about half of 3.3 V, or 1.65 V.

ADC acquisition

1. Open `/dev/i2c-1` and select address `0x48`.
2. Configure a single-ended, single-shot conversion on channel 3.
3. Use the 6.144 V full-scale range.
4. Wait up to 100 ms for completion.
5. Decode ADS1015 12-bit or ADS1115 16-bit data.

Resistance

For the default arrangement, where the thermistor is connected to ground:

$$R_T = R_F \times V_O / (V_S - V_O)$$

If the thermistor is connected to the supply instead:

$$R_T = R_F \times (V_S - V_O) / V_O$$

6. Temperature Calibration

Beta equation

$$1 / T = 1 / T_0 + [\ln(R_T / R_0) / \beta]$$

Temperatures in the beta equation are kelvin. The calculated value is converted to Fahrenheit:

$$T_F = (T_K - 273.15) \times 9 / 5 + 32$$

Current defaults

Setting	Default
Divider supply	3.3 V
Fixed resistor	10 kΩ
Nominal thermistor resistance	10 kΩ
Nominal temperature	25°C
Beta	3950 K
Thermistor position	Analog 1 to ground

With these defaults, 1.65 V maps to 77.0°F and 1.70 V maps to approximately 74.6°F.

Improving accuracy

- Measure the actual divider supply voltage.
- Measure the fixed resistor rather than relying only on its marked tolerance.
- Use the thermistor datasheet's nominal resistance and beta value.
- For a broad temperature range, consider thermistor-specific Steinhart-Hart coefficients.

7. Error Handling and Troubleshooting

Symptom	Check
GPIO line is busy	Close any other Pi Panel instance.
Analog 1 unavailable	Confirm <code>/dev/i2c-1</code> , group access, and address <code>0x48</code> with <code>i2cdetect -y 1</code> .
Temperature unavailable	Check that voltage is strictly between 0 V and the configured divider supply.
Temperature is inaccurate	Verify supply, resistor, beta, nominal resistance, and divider orientation.
Button triggers unpredictably	Confirm Input 1 has an external pull-down and is not floating.
Pi desktop window does not open	Confirm the graphical desktop is logged in as <code>bill</code> and use the local debug profile.
Forwarded window does not open	Start XLaunch, connect PuTTY with forwarding, and match <code>DISPLAY</code> .

Failure policy

ADC and conversion errors are exposed in the window without stopping LED control. GPIO polling errors are caught in the timer callback and exit the application. Normal exit turns the LED off.

8. Configuration Reference

Environment variable	Default	Purpose
<code>PI_PANEL_I2C_DEVICE</code>	<code>/dev/i2c-1</code>	Linux I2C device path.
<code>PI_PANEL_DIVIDER_VOLTS</code>	<code>3.3</code>	Divider supply voltage.
<code>PI_PANEL_FIXED_RESISTOR_OHMS</code>	<code>10000</code>	Fixed resistor value.
<code>PI_PANEL_THERMISTOR_NOMINAL_OHMS</code>	<code>10000</code>	Resistance at nominal temperature.
<code>PI_PANEL_THERMISTOR_NOMINAL_C</code>	<code>25</code>	Nominal temperature in Celsius.
<code>PI_PANEL_THERMISTOR_BETA</code>	<code>3950</code>	Beta value in kelvin.
<code>PI_PANEL_THERMISTOR_TO_GROUND</code>	<code>1</code>	Set to <code>0</code> when the thermistor is connected to the supply.

Hardware constants

Function	Mapping
LED	Explorer HAT LED 2 / BCM GPIO 17
Button	Explorer HAT Input 1 / BCM GPIO 23 / active-high
Thermistor voltage	Explorer HAT Analog 1 / ADS1015 channel 3
ADC	I2C address <code>0x48</code> , 6.144 V full-scale
GPIO chip	<code>/dev/gpiochip0</code>

Source references: `Controllers/` , `ViewModels/` , `Main.qml` , `main.cpp` , and `tests/` .